

JSON Fundamentals: Data Interchange in JavaScript

Introduction to JSON

JSON (JavaScript Object Notation) is a lightweight, text-based data interchange format that is easy for humans to read and write, and easy for machines to parse and generate. Despite its name deriving from JavaScript, JSON is language-independent and has become the universal standard for data exchange in modern web applications, APIs, and configuration files.

What is JSON?

JSON is a format for representing structured data as text. It uses a syntax that is a subset of JavaScript object literal notation, which makes it particularly convenient for JavaScript developers. However, JSON parsers and generators exist for virtually every programming language, making it an ideal choice for cross-platform data exchange.

Key Characteristics

- **Language-independent:** While derived from JavaScript, JSON can be used with any programming language
- **Text-based:** JSON data is stored as plain text, making it easily transmittable over networks
- **Lightweight:** Minimal syntax overhead compared to alternatives like XML
- **Self-describing:** The structure is clear from the data itself
- **Hierarchical:** Supports nested data structures

JSON Data Types

JSON supports six fundamental data types:

1. String

A sequence of Unicode characters enclosed in double quotes.

```
{  
  "name": "Aditi",  
  "city": "Mumbai",  
  "message": "Hello, World!"  
}
```

Output:

String Rules:

- Must use double quotes (single quotes are invalid in JSON)
- Can contain escaped characters: \n (newline), \t (tab), \" (quote), \\ (backslash)
- Unicode characters are supported: "नमस्ते" or "\u0928\u092E\u0938\u094D\u0924\u0947"

2. Number

Integer or floating-point values. JSON does not distinguish between different numeric types.

```
{  
  "age": 45,  
  "temperature": -12.5,  
  "scientificNotation": 1.5e10,  
  "percentage": 99.99  
}
```

Output:

Number Rules:

- No leading zeros (015 is invalid, use 15)
- Can be negative
- Can use scientific notation (1.5e10 = 15000000000)

- No special values like NaN or Infinity (these must be represented as strings or null)

3. Boolean

Represents true or false values.

```
{
  "isActive": true,
  "hasAccess": false,
  "emailVerified": true
}
```

Output:

Boolean Rules:

- Only two values: `true` or `false`
- Must be lowercase (True, TRUE, or False are invalid)

4. Null

Represents an empty or non-existent value.

```
{
  "middleName": null,
  "spouse": null,
  "previousAddress": null
}
```

Output:

Null Rules:

- Must be lowercase
- Represents intentional absence of a value
- Different from undefined (which doesn't exist in JSON)

5. Object

An unordered collection of key-value pairs enclosed in curly braces.

```
{  
  "person": {  
    "name": "Aditi",  
    "age": 45,  
    "address": {  
      "street": "Some street",  
      "city": "Some city",  
      "pincode": "400001"  
    }  
  }  
}
```

Output:

Object Rules:

- Keys must be strings enclosed in double quotes
- Key-value pairs separated by colons
- Multiple pairs separated by commas
- Can be nested indefinitely
- No trailing comma after the last property

6. Array

An ordered list of values enclosed in square brackets.

```
{
  "colors": ["red", "green", "blue"],
  "numbers": [1, 2, 3, 4, 5],
  "mixed": [42, "hello", true, null, {"key": "value"}],
  "matrix": [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
  ]
}
```

Output:

Array Rules:

- Values separated by commas
- Can contain any JSON data type, including other arrays and objects
- Can be empty: []
- Maintains order of elements
- No trailing comma after the last element

JavaScript Objects vs JSON

While JSON syntax is derived from JavaScript object literals, they are not identical. Understanding the differences is crucial:

Feature	JavaScript Object	JSON
Keys	Can be unquoted	Must be quoted strings

Feature	JavaScript Object	JSON
String quotes	Single or double quotes	Only double quotes
Data types	Functions, Date, undefined, Symbol, etc.	Only: string, number, boolean, null, object, array
Trailing commas	Allowed (in modern JS)	Not allowed
Comments	Allowed	Not allowed
Format	Code/Data structure	Pure data (text string)

Example Comparison

```
// JavaScript Object (valid in JS, invalid as JSON)
const jsObject = {
  name: 'Aditi', // Single quotes, unquoted key
  age: 45,
  greet: function() { // Functions not allowed in JSON
    return "Hello";
  },
  timestamp: new Date(), // Date objects not allowed in JSON
  value: undefined, // undefined not allowed in JSON
};

// Valid JSON (as a string)
const jsonString = `{
  "name": "Aditi",
  "age": 45,
  "timestamp": "2026-01-19T10:30:00Z",
  "value": null
}`;
```

Output:

Key Insight: In JavaScript, objects are in-memory data structures. JSON is a text format used for data interchange. You must convert between them using `JSON.stringify()` and `JSON.parse()`.

Working with JSON in JavaScript

Creating a JavaScript Object

```
const person = {  
  name: "Aditi",  
  age: 45,  
  address: {  
    street: "Some street",  
    city: "Some city"  
  }  
};
```

Output:

Console Output:

```
{  
  name: "Aditi",  
  age: 45,  
  address: {  
    street: "Some street",  
    city: "Some city"  
  }  
}
```

This is a JavaScript object stored in memory. You can access properties using dot notation (`person.name`) or bracket notation (`person['name']`).

JSON.stringify() - Object to JSON String

The `JSON.stringify()` method converts a JavaScript object into a JSON string. This is essential when you need to send data to a server, save it to a file, or store it in local storage.

Basic Usage

```
const person = {  
  name: "Aditi",  
  age: 45,  
  address: {  
    street: "Some street",  
    city: "Some city"  
  }  
};  
  
const jsonString = JSON.stringify(person);  
console.log(jsonString);
```

Output:

Console Output:

```
{"name":"Aditi","age":45,"address":{"street":"Some street","city":"Some city"}}
```

Notice that the output is a single-line string with:

- All keys enclosed in double quotes
- No extra whitespace
- The entire structure as one continuous string

Checking the Type

```
const person = { name: "Aditi", age: 45 };  
const jsonString = JSON.stringify(person);  
  
console.log(typeof person);  
console.log(typeof jsonString);
```


Output:**Console Output:**

```
object  
string
```

This demonstrates the fundamental difference: `person` is an object, while `jsonString` is a text string representation of that object.

Pretty Printing with Indentation

`JSON.stringify()` accepts optional parameters for formatting. The second parameter is a replacer function (often null), and the third is the number of spaces for indentation.

```
const person = {  
  name: "Aditi",  
  age: 45,  
  address: {  
    street: "Some street",  
    city: "Some city"  
  },  
  hobbies: ["reading", "traveling"]  
};  
  
const prettyJSON = JSON.stringify(person, null, 2);  
console.log(prettyJSON);
```

Output:**Console Output:**

```
{  
  "name": "Aditi",  
  "age": 45,  
  "address": {  
    "street": "Some street",
```

```
    "city": "Some city"
  },
  "hobbies": [
    "reading",
    "traveling"
  ]
}
```

The third parameter (2) adds 2-space indentation, making the JSON human-readable. This is useful for debugging and logging.

Selective Stringification with Replacer

The second parameter can filter which properties are included in the JSON string.

```
const user = {
  username: "aditi_sharma",
  password: "secret123",
  email: "aditi@example.com",
  age: 45
};

// Only include specific properties
const safeJSON = JSON.stringify(user, ["username", "email", "age"], 2);
console.log(safeJSON);
```

Output:

Console Output:

```
{
  "username": "aditi_sharma",
  "email": "aditi@example.com",
  "age": 45
}
```

The password field is excluded, making this safe for logging or sending to a client. This is crucial for security when handling sensitive data.

Custom Replacer Function

```
const product = {
  name: "Laptop",
  price: 75000,
  discount: 5000,
  stock: 50
};

const jsonString = JSON.stringify(product, (key, value) => {
  // Hide stock information
  if (key === "stock") {
    return undefined;
  }
  // Round price values
  if (key === "price" || key === "discount") {
    return Math.round(value);
  }
  return value;
}, 2);

console.log(jsonString);
```

Output:

Console Output:

```
{
  "name": "Laptop",
  "price": 75000,
  "discount": 5000
}
```

The replacer function is called for each property. Returning `undefined` excludes the property. You can also transform values as needed.

Handling Special Cases

```
const data = {
  validString: "Hello",
  validNumber: 42,
  validBoolean: true,
  validNull: null,
  undefinedValue: undefined, // Will be omitted
  functionValue: function() { return "test"; }, // Will be omitted
  dateValue: new Date(), // Converted to ISO string
  symbolValue: Symbol("id"), // Will be omitted
  infinityValue: Infinity // Converted to null
};

console.log(JSON.stringify(data, null, 2));
```

Output:

Console Output:

```
{
  "validString": "Hello",
  "validNumber": 42,
  "validBoolean": true,
  "validNull": null,
  "dateValue": "2026-01-19T10:30:00.000Z",
  "infinityValue": null
}
```

Important Behavior:

- `undefined`, functions, and Symbols are omitted from objects
- Dates are converted to ISO 8601 strings

- `Infinity` and `NaN` become `null`
- In arrays, these values become `null` instead of being omitted

JSON.parse() - JSON String to Object

The `JSON.parse()` method converts a JSON string back into a JavaScript object. This is used when receiving data from a server, reading from a file, or retrieving from storage.

Basic Usage

```
const jsonString = '{"name":"Aditi","age":45,"city":"Mumbai"}';

const person = JSON.parse(jsonString);

console.log(person);
console.log(person.name);
console.log(person.age);
```

Output:

Console Output:

```
{ name: 'Aditi', age: 45, city: 'Mumbai' }
Aditi
45
```

The JSON string is converted to a JavaScript object, and you can now access its properties using standard object notation.

Parsing Nested Structures

```
const jsonString = `{
  "name": "Aditi",
  "age": 45,
  "address": {
    "street": "Some street",
    "city": "Some city",
    "pincode": "400001"
  },
  "hobbies": ["reading", "traveling", "photography"]
}`;

const person = JSON.parse(jsonString);

console.log(person.address.city);
console.log(person.hobbies[0]);
console.log(person.hobbies.length);
```

Output:

Console Output:

```
Some city
reading
3
```

Nested objects and arrays are fully parsed and accessible. You can navigate through the structure using dot notation and array indices.

Reviver Function for Custom Parsing

The second parameter of `JSON.parse()` is a reviver function that allows you to transform values during parsing.

```
const jsonString = `{
  "name": "Product Launch",
  "date": "2026-01-19T10:30:00.000Z",
  "price": "75000",
  "active": "true"
}`;

const data = JSON.parse(jsonString, (key, value) => {
  // Convert date strings to Date objects
  if (key === "date") {
    return new Date(value);
  }
  // Convert price strings to numbers
  if (key === "price") {
    return parseFloat(value);
  }
  // Convert string booleans to actual booleans
  if (value === "true") return true;
  if (value === "false") return false;

  return value;
});

console.log(data.date);
console.log(data.date instanceof Date);
console.log(data.price);
console.log(typeof data.price);
console.log(data.active);
console.log(typeof data.active);
```

Output:

Console Output:

```
2026-01-19T10:30:00.000Z
true
75000
number
true
boolean
```

The `reviver` function transforms string values into their appropriate types. This is particularly useful when working with APIs that return dates or numbers as strings.

Error Handling with `JSON.parse()`

`JSON.parse()` throws a `SyntaxError` if the string is not valid JSON. Always use try-catch when parsing untrusted input.

```
const invalidJSON = `{
  name: "Aditi", // Missing quotes around key
  'age': 45, // Single quotes not allowed
  city: "Mumbai", // Trailing comma on next line
}`;

try {
  const data = JSON.parse(invalidJSON);
  console.log(data);
} catch (error) {
  console.log("Parsing failed!");
  console.log("Error:", error.message);
}
```

Output:

Console Output:

```
Parsing failed!
Error: Unexpected token n in JSON at position 6
```

Common JSON syntax errors include:

- Unquoted keys
- Single quotes instead of double quotes
- Trailing commas

- Missing commas between properties
- Comments (not allowed in JSON)

Safe Parsing with Validation

```
function safeJSONParse(jsonString, defaultValue = null) {  
  try {  
    return JSON.parse(jsonString);  
  } catch (error) {  
    console.error("JSON parsing error:", error.message);  
    return defaultValue;  
  }  
}  
  
const validJSON = '{"name":"Aditi","age":45}';  
const invalidJSON = '{name:"Aditi"}';  
  
const result1 = safeJSONParse(validJSON);  
const result2 = safeJSONParse(invalidJSON, { error: true });  
  
console.log("Valid result:", result1);  
console.log("Invalid result:", result2);
```

Output:

Console Output:

```
JSON parsing error: Unexpected token n in JSON at position 1  
Valid result: { name: 'Aditi', age: 45 }  
Invalid result: { error: true }
```

This helper function provides graceful error handling, returning a default value instead of crashing when JSON is invalid.

Practical Use Cases

1. API Communication

```
// Sending data to server
const userData = {
  username: "aditi_sharma",
  email: "aditi@example.com",
  age: 45
};

fetch('https://api.example.com/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(userData)
})
.then(response => response.json()) // Parses JSON response
.then(data => {
  console.log('User created:', data);
});
```

Output:

Workflow:

1. JavaScript object is converted to JSON string with `JSON.stringify()`
2. JSON string is sent in the HTTP request body
3. Server processes the request and sends JSON response
4. `response.json()` parses the JSON string back to a JavaScript object
5. You can now work with the response data as a regular object

2. Local Storage

```
// Storing complex data in localStorage
const userPreferences = {
  theme: "dark",
  language: "en",
  notifications: {
    email: true,
    push: false,
    sms: true
  },
  recentSearches: ["JavaScript", "JSON", "HTTP"]
};

// Save to localStorage (requires JSON string)
localStorage.setItem('preferences', JSON.stringify(userPreferences));

// Retrieve from localStorage
const storedPreferences = localStorage.getItem('preferences');
const preferences = JSON.parse(storedPreferences);

console.log(preferences.theme);
console.log(preferences.notifications.email);
console.log(preferences.recentSearches[0]);
```

Output:

Console Output:

```
dark
true
JavaScript
```

Note: localStorage can only store strings. `JSON.stringify()` converts objects to strings for storage, and `JSON.parse()` converts them back when retrieving.

3. Deep Cloning Objects

```
const original = {
  name: "Aditi",
  age: 45,
  address: {
    city: "Mumbai"
  },
  hobbies: ["reading", "traveling"]
};

// Shallow copy - changes to nested objects affect original
const shallowCopy = { ...original };
shallowCopy.address.city = "Delhi";
console.log("Original city:", original.address.city);

// Deep copy using JSON
const deepCopy = JSON.parse(JSON.stringify(original));
deepCopy.address.city = "Bangalore";
console.log("Original city after deep copy:", original.address.city);
console.log("Deep copy city:", deepCopy.address.city);
```